

TEAM ONBOARDING · 기술 문서

YGT 2025

코드 구조 · 시스템 아키텍처 · 모듈 레퍼런스

공군사관학교 2학년 생도 서바이벌 미니게임. 이 문서는 팀원이 코드베이스에 빠르게 합류할 수 있도록 **디렉토리 역할**, **게임을 이루는 구성요소**, **전체 아키텍처**, **각 모듈의 책임**을 도표와 함께 정리한 온보딩 가이드입니다.

Phaser 3

TypeScript

Vite · PWA

Cloudflare Pages

Pages Functions

D1 (SQLite)

01 프로젝트 개요

무엇을 만드는가 — 그리고 이 문서를 어떻게 읽을 것인가.

YGT 2025는 “몰래 하다가 들키기 직전에 멈춰라”를 핵심 메커닉으로 하는 **탐지 회피 + 반응·리듬·운빨** 미니게임 모음입니다. 하루에 메인 퀘스트 하나를 수행하고, 성공하면 다음 날로, 발각/HP 0/미니 퀘스트 실패 시 게임 오버 후 생존 일수를 온라인 리더보드에 등록합니다.

기술 스택 한눈에

레이어	기술	역할
게임 엔진	Phaser 3	씬·입력·렌더링·아케이드 물리. 720×1280 세로 캔버스를 FIT 스케일로 표시
언어	TypeScript (strict)	클라이언트 + 서버 함수 모두 정적 타입. 빌드 시 tsc --noEmit로 검증
번들러	Vite + vite-plugin-pwa	개발 서버·프로덕션 번들·서비스워커/매니페스트 생성
배포·호스팅	Cloudflare Pages	GitHub 연동 자동 배포, 커스텀 도메인 자동 HTTPS
서버리스 API	Pages Functions	/api/leaderboard 리더보드 조회·등록 (외부 BaaS 불필요)
데이터베이스	Cloudflare D1	SQLite 기반. 리더보드 저장 + 정렬 인덱스
저장(클라)	localStorage	진행 상태 저장 + 리더보드 오프라인 등록 큐
사운드	Web Audio API	칩튠 BGM·SFX를 코드로 합성 (master → bgm/sfx 버스)
아트	Gemini 이미지 생성	손그림체 캐릭터·배경 PNG 생성 → 후처리 → 프리로드

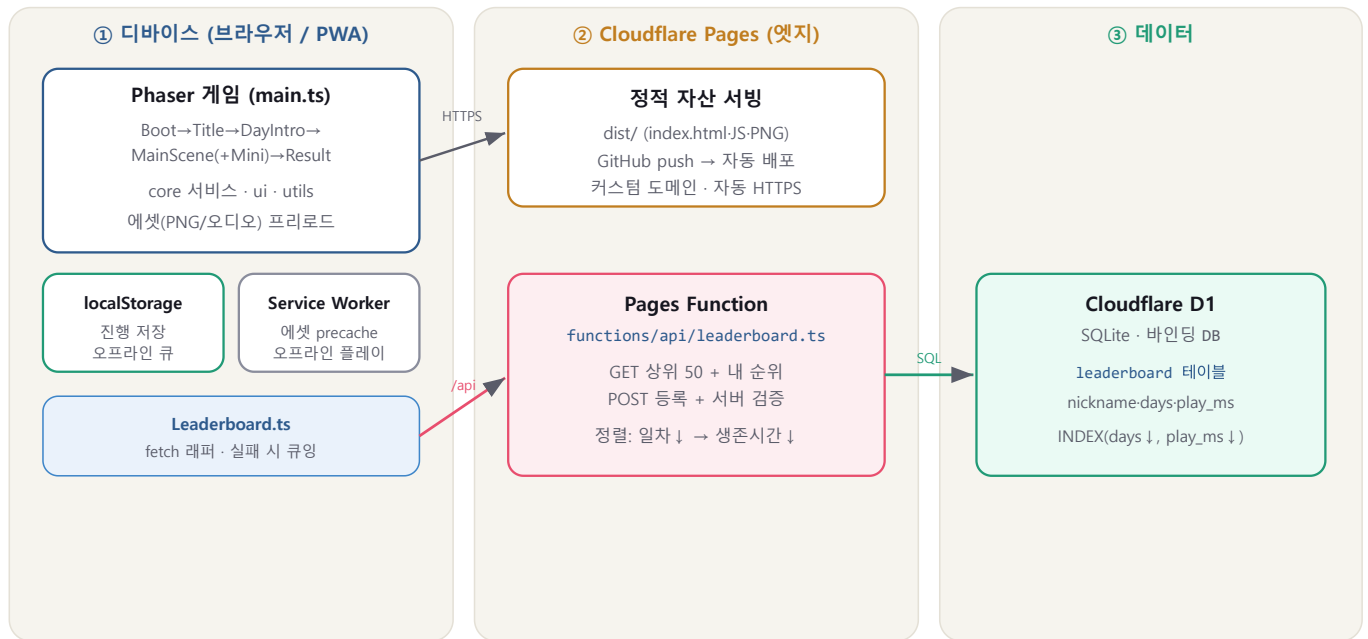
개발 제약: 개발 PC는 Windows이고 Mac-USB를 쓸 수 없어 네이티브 앱 빌드가 없습니다. **순수 웹 PWA**로만 개발하며, 배포본을 폰에서 “홈 화면에 추가”해 실행 테스트합니다.

구성요소(빅 픽처)

- **게임 클라이언트** — Phaser 씬 그래프 + 3개의 전역 싱글턴 서비스(상태·퀘스트·오디오) + UI 컴포넌트 + 에셋
- **엡지 백엔드** — Cloudflare Pages가 정적 파일을 서빙하고, 같은 프로젝트의 Pages Function이 리더보드 API를 담당
- **데이터** — 서버측은 D1(리더보드), 클라이언트측은 localStorage(진행 저장 + 오프라인 큐)

02 시스템 아키텍처

브라우저 안의 게임부터 엣지의 서버리스 함수·DB까지 — 하나의 Cloudflare 프로젝트로.



▲ 게임·서버·DB가 하나의 Cloudflare Pages 프로젝트 안에서 해결된다. 리더보드를 뺀 나머지는 오프라인 동작.

왜 이 구조인가

- **서버 운영 부담 최소화** — 정적 호스팅 + 서버리스 함수 + 관리형 SQLite. 상시 켜진 서버가 없습니다.
- **같은 오리진** — 클라이언트는 /api/...를 같은 도메인에서 호출하므로 CORS 설정이 필요 없습니다.
- **오프라인 우선** — 서비스워커가 에셋을 precache해 설치 후 오프라인 플레이 가능. 리더보드만 온라인이 필요하며, 실패 시 localStorage 큐로 재시도합니다.

03 디렉토리 구조 & 역할

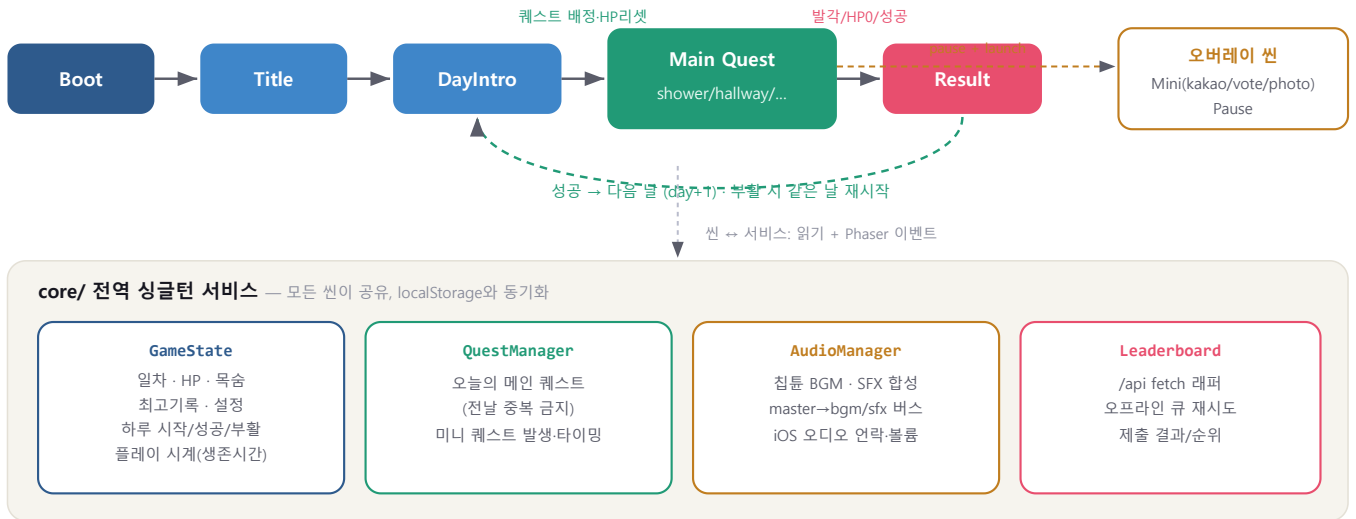
저장소를 열었을 때 “어디에 무엇이 있는지” — 폴더 단위 지도.

```
ygt2025/ └─ index.html # 진입 HTML · 카톡 인앱 감지/리다이렉트 └─ vite.config.ts # Vite + PWA(서비스워커/매니페스트)
설정 └─ wrangler.toml # Cloudflare · D1 바인딩(DB) └─ src/ # ▶ 게임 클라이언트 소스 (아래 $4~$7) | └─ main.ts #
Phaser Game 부팅 · 씬 등록 · 뷰포트/오디오/가시성 | └─ config.ts # ★ 밸런스 상수·함수 단일 지정(난이도/확률/시간/색/폰
트) | └─ types.ts # 공용 타입 정의 | └─ core/ # 전역 서비스 싱글턴 (상태·퀘스트·오디오·리더보드) | └─ scenes/ # Phaser
씬 (부팅/타이틀/하루/결과 + main/ + mini/) | └─ ui/ # 재사용 UI·캐릭터·배경·카톡 조각 | └─ utils/ # RNG · 가상 키보드 ·
햅틱 · 닉네임 다이얼로그 | └─ assets/ # manifest(IMAGE_KEYS) · 사용자 제공 에셋 자리 | └─ functions/ # ▶ Cloudflare
Pages Functions (서버리스) | └─ api//leaderboard.ts # GET 조회 / POST 등록 + 서버 검증 | └─ migrations/ # D1 스키마 마
이그레이션(0001 테이블 · 0002 인덱스) | └─ public/ # 정적 자원 (그대로 복사됨) | └─ assets/img/ # 게임 PNG(캐릭터/배경/소
품/문) – BootScene가 프리로드 | └─ audio/ # 실음원(샤워 노래·물소리·전자레인지·수다) | └─ icons/ # PWA 아이콘 192/512 ·
마스크러블 | └─ scripts/ # gen-icons.mjs (PWA 아이콘 생성) | └─ dist/ # 빌드 산출물(배포 대상) | └─ reference/ # 참고 스샷/실
사진 (.gitignore – 3rd-party IP)
```

폴더	역할 요약
src/core/	게임 전역을 관통하는 싱글턴 서비스. 씬에 종속되지 않는 상태·규칙·오디오·네트워크. 씬은 이들을 읽고 이 벤트로만 통신.
src/scenes/	화면 단위 Phaser 씬. 공통 로직은 BaseMainScene/BaseMiniScene에 모으고 각 퀘스트가 상속.
src/ui/	씬들이 공유하는 표현 계층 — 캐릭터 스프라이트(Cadet), 배경·소품 헬퍼, 카톡 UI 조각, 버튼/바/패널.
src/utils/	도메인 독립 헬퍼 — 확률 RNG, 모바일 가상 키보드용 숨은 input, 진동, 닉네임 입력.
functions/	엡지에서 도는 서버리스 API. 게임 코드와 분리되어 D1에 접근하고 기록을 검증.
public/	변들을 거치지 않고 그대로 서빙되는 자원(게임 PNG·실음원·PWA 아이콘·매니페스트).
migrations/	D1 스키마 버전 관리. wrangler d1 migrations apply로 로컬/원격에 적용.

04 런타임 아키텍처 — 씬 흐름 & 전역 서비스

게임이 실제로 도는 방식: 씬 그래프가 화면을 전환하고, 싱글턴 서비스가 상태를 들고 있다.



▲ 씬은 화면 전환만 담당하고, 상태·규칙·오디오·네트워크는 core/ 싱글턴이 소유한다. 씬 간 통신은 GameState + 이벤트로 한정.

설계 원칙: 밸런스 수치(확률·시간·HP)는 씬 코드에 매직넘버로 두지 않고 **전부 config.ts**에 상수/함수로 모읍니다. 씬 간 통신은 **GameState 싱글턴 + Phaser 이벤트**만 사용합니다.

05 씬 계층 & 구성요소

공통 로직은 베이스 씬에 모으고, 각 퀘스트는 얇게 상속한다.

씬 목록

씬 (key)	분류	책임
Boot	부팅	PNG·오디오 에셋 프리로드 + 로딩바
Title	메뉴	타이틀 · 게임시작/리더보드/게임설명/닉네임설정
DayIntro	연출	“N일차” 스플래시 → HP 리셋 → 오늘의 퀘스트 배정(자동 진행)
Result	연출	성공/부활/게임오버 분기 · 신기록 시 리더보드 등록
Pause	오버레이	일시정지·소리설정 · 3-2-1 재개 카운트다운
BaseMainScene	추상	메인 퀘 공통: HP/목숨 바 · 선배 등장 루프 · 미니 인터럽트 · 성공/실패/데미지
main/ ×5	메인 퀘	shower · hallway · microwave · walk · wallpunch
BaseMiniScene	추상	미니 퀘 공통: 카톡 셀 레이아웃 · 종료 후 본 씬 resume
mini/ ×3	미니 퀘	kakao(타자) · vote(함정 선지) · photo(사진 고르기/틀린그림찾기)

메인 퀘스트가 상속으로 얻는 것 (BaseMainScene)

- **선배 등장 스케줄러** — 예고 없이 등장→체류→퇴장 반복. 패턴(연타/페인트/뜸들이기)으로 예측을 흔들되, **최소 회복시간·점유율 상한**으로 “운빨 사망”을 수학적으로 차단.
- **미니 퀘스트 인터럽트** — 랜덤 타이밍에 본 씬 pause → 미니 씬 launch → 복귀 시 resume.
- **공통 결과 처리** — succeed() / fail() / failCaught()(클로즈업 연출) / applyDamage().
- **공통 UI** — HP 바·목숨 바·소리설정·일시정지·위험 비네트 + 연습 모드 배지.

메인 퀘스트 5종

- **shower** — 노래 자동 재생, 선배 등장 시 Ⅱ 홀드로 숨죽이기
- **hallway** — 건장 읽어 인사/경례 (박자 판정 리듬)
- **microwave** — 조리 게이지 채우기, 세탁실로 은신
- **walk** — 톱다운 잠입, 시야 피해 걷기/뛰기
- **wallpunch** — 노트 높이 탭 리듬 (10일차대 등장)

미니 퀘스트 3종 (돌발)

- **kakao** — 제한시간 내 문장 타이핑 (가상 키보드)
- **vote** — 이중 부정문 함정 선지 독해
- **photo** — 정리된 관물대 고르기 / 틀린 부분 찾기

세 미니 퀘스트는 ui/kakao.ts 공용 모듈로 실제 카카오톡 UI를 공유한다.

06 모듈 레퍼런스

파일 단위 책임 — 어디를 고쳐야 하는지 찾는 색인.

core/ — 전역 서비스

모듈	책임
core/GameState.ts	일차·HP(하루 리셋)·목숨(1/5 단위)·최고기록·설정을 들고 localStorage와 동기화. newRun/startDay/completeDay/gameOver/tryRevive와 생존시간 측정용 플레이 시계(hold/release).
core/QuestManager.ts	오늘의 메인 퀘스트 선택(전날 중복 금지, 일정 일차부터 베키기 편입) + 미니 퀘스트 발생 확률·타이밍 계획.
core/AudioManager.ts	Web Audio로 칩튠 BGM·SFX를 합성. master→bgm/sfx 버스 볼륨, iOS 오토언락(무음 루프), 노래/환경음 제어.
core/Leaderboard.ts	/api/leaderboard GET/POST 래퍼. 네트워크 실패 시 localStorage 큐에 저장 후 다음 접속에 재시도.

ui/ — 표현 계층

모듈	책임
ui/Characters.ts	Cadet 스프라이트 (이미지 기반): setMotion/setFace/saluteOnce/punchOnce. 계급 견장은 코드가 직접 렌더 (buildRankBoards)해 개수를 보장. 말풍선 헬퍼 포함.
ui/Scenery.ts	배경·소품 배치 헬퍼(addSceneBg/addProp). 원근(소실점) 기반 스케일 배치.
ui/kakao.ts	실제 카카오톡 UI 조각(상태바·말풍선·입력바·상세보기 페이지). 세 미니 퀘스트가 공유.
ui/Button.ts · HpBar.ts · LivesBar.ts	버튼(하드 색도 · 눌림 애니), HP 바, 목숨 하트 바(부분 채움).
ui/DialogueBox.ts · HelpPanel.ts · LeaderboardPanel.ts · VolumePanel.ts	미연시풍 대사창, 게임설명, 리더보드 패널, 볼륨 설정 패널.

utils/ · 최상위

모듈	책임
utils/mobileInput.ts	캔버스 위에 겹치는 숨은 <input> — 타자 미니 퀘스트용 모바일 가상 키보드(카톡 입력바 모양).
utils/rng.ts · haptics.ts · nicknameDialog.ts	확률/랜덤 헬퍼, 진동 패턴, 닉네임 입력 다이얼로그.
config.ts	★ 튜닝의 단일 지점 — 난이도 곡선·조우 빈도·제한시간·확률·퀘스트별 파라미터·색·폰트·QUEST_META. 개별 값을 만지기 전에 전역 손잡이 DIFF_SCALE부터.
assets/manifest.ts	IMAGE_KEYS — BootScene가 프리로드할 PNG 목록. 에셋 추가/삭제 시 이곳과 public/assets/img를 반드시 대조.
main.ts	Phaser Game 생성·씬 등록, iOS 뷰포트 안정화, 오디오 오토언락, 백그라운드 전환 시 자동 일시정지.

07 데이터 흐름 · 에셋 파이프라인 · 워크플로

기록은 어떻게 저장되고, 그림은 어떻게 게임에 들어오며, 코드는 어떻게 배포되는가.

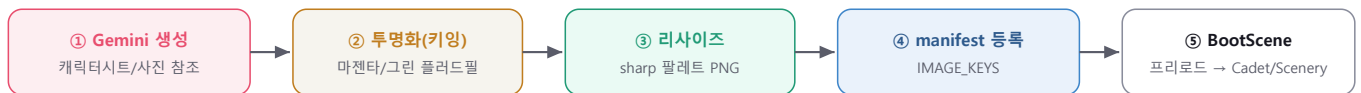
리더보드 데이터 흐름



정렬 규칙 — 일차 DESC → 같은 일차면 생존시간 play_ms DESC(오래 버틴 순). 닉네임별 최고 기록 1건만 노출(윈도우 함수). **서버 검증** — 닉네임 길이·금칙어, 일차 범위, 최소/최대 플레이시간으로 비정상 기록 차단.

```
leaderboard(id, nickname, days, play_ms, created_at) · INDEX(days DESC, play_ms DESC)
```

에셋 파이프라인 (아트)



주의: 매니페스트 누락 = Phaser의 “초록 테두리 검정 박스” 플레이스홀더. 에셋을 추가/삭제하면 IMAGE_KEYS와 public/assets/img를 반드시 대조하세요. 견장 줄 수는 생성 이미지가 아니라 **코드가 그린 것이** 정답입니다.

개발 · 빌드 · 배포 워크플로

명령 / 단계	설명
<code>npm run dev</code>	Vite 개발 서버(--host) — 같은 Wi-Fi의 폰 브라우저에서 접속해 테스트
<code>npm run build</code>	strict TS 체크(클라이언트 + Functions) → 프로덕션 번들 + PWA 생성
<code>npm run pages:dev</code>	빌드 산출물 + Functions + 로컬 D1(미니플레이어) 동시 서빙
<code>d1:migrate:local / :remote</code>	D1 스키마 마이그레이션 적용(로컬/원격)
<code>git push (main)</code>	Cloudflare Pages CI가 자동 빌드·배포. PR 브랜치는 프리뷰 URL 생성(폰 테스트용)
헤드리스 검증	build → preview → puppeteer-core + 시스템 크롬으로 씬별 스크린샷 자동 캡처(9:16 뷰포트)

YGT 2025 · 팀 온보딩 기술 문서 — 코드가 곧 최신 사실입니다. 밸런스는 config.ts, 상태는 core/, 화면은 scenes/부터 보세요.